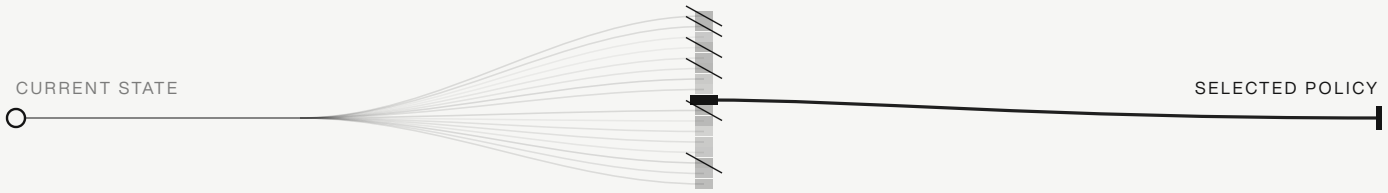
WHAT AURELIUS OPTIMIZES

Everyone optimizes the next decision. Aurelius optimizes the fleet's trajectory.

FORK · SIMULATE UNDER CONSTRAINTS · SELECT ONE



WHAT IT IS

It is not a forecaster and not a scheduler replacement. It is a **supervisory layer** that sits above Kubernetes, Slurm, or a serving router and never touches payloads. Each control period it forks the current fleet state into many counterfactual trajectories, applies coupled policies across the controls the stack already exposes, advances each trajectory under SLA, capacity, and power constraints, kills the ones that violate, and returns one policy for the existing stack to execute.

THE FINDING

Exactly one thing changed. Holding the forecasts, the objective, the simulator, and the constraints fixed, only the planner's representable policies and search varied, so any change in the result is caused by the **search** and nothing else. A single coherent policy already beats a reactive scheduler several times over. Searching a fixed set of policies saturates quickly. The largest block of value appears only when the search can represent **coupled policies that no fixed grid contains**.

If that generalizes, it is a different design axis, not a tuning win.

The full technical report carries the numbers, the production-class baseline, and the method. This page is only the idea.